COMPARAR LA IMPLEMENTACIÓN DE UN DICCIONARIO DE DATOS FÍSICO EN DIFERENTES SGBD

INTEGRANTES:

SONIA MARIA GOMEZ BOLIVAR
CAMILA ANDREA GOMEZ ORTIZ

PROGRAMA:

TECNOLOGÍA EN DESARROLLO DE

SOFTWARE G100 (SD1006) BASES

DE DATOS I

DOCENTE:

ORALIA CORTES
GRAJALES

INSTITUCIÓN UNIVERSITARIA

PASCUAL BRAVO MEDELLÍN,

COLOMBIA

2026-2

INTRODUCCIÓN

El presente trabajo describe el proceso de diseño de implementación del modelo físico de datos para la red social pascualina. El principal propósito es construir una infraestructura tecnológica robusta que dé solución a los problemas de comunicación de los estudiantes en la actualidad, esto va a facilitar la creación de perfiles, la organización de grupos de estudio, la publicación de información académica y la coordinación de eventos informales de forma rápida y segura.

A diferencia de los modelos relacionales tradicionales que no son flexibles y estructurados, esta implementación se orienta en la adaptación y la evolución de la base de datos mediante el uso de tecnologías hibridas. Para lograrlo, se seleccionó el Sistema Gestor de Bases de Datos PostgreSQL, aprovechando sus capacidades nativas para el procesamiento de datos relacionales estándar y su compatibilidad avanzada con estructuras de datos semiestructurados como JSON y JSONB. Este enfoque híbrido permite combinar la seguridad y consistencia de las restricciones de integridad y las claves foráneas con la flexibilidad del paradigma NoSQL, el cual es indispensable hoy en día para dar solución a retos modernos de Big Data e Internet de las Cosas (IoT).

OBJETIVO

Diseñar la estructura física de la base de datos
Implementar el modelo en tres SGBD
Incorporar datos semi estructurados mediante JSON
Documentar el proceso en diccionarios de datos

1. JUSTIFICACION DE LOS SISTEMAS GESTORES DE BASES DE DATOS SGBD

En el ambiente moderno de desarrollo de software, las plataformas no solo se limitan a almacenar textos cortos o valores numéricos. Una red social universitaria se enfrenta en un flujo continuo de interacciones diversas que saturan los esquemas relacionales clásicos. Para dar cumplimientos a esta rúbrica de evaluación, se diseñó e integro estratégicamente el manejo de datos semiestructurados bajo los siguientes pilares tecnológicos:

Caso de Uso 1: Internet de las Cosas (IoT) y Geolocalización en Tiempo Real de Eventos
Los estudiantes coordinan constantemente tutorías académicas, clubes de interés en distintas zomas del campus y hackatones. Para soportar esto en la tabla evento incluye un campo semiestructurado nativo denominado geolocalización. En un escenario productivo, este campo actúa como sumidero (sink) de datos provenientes de la telemetría de dispositivos móviles. El formato alberga las coordenadas exactas extraídas de sensores GPS (latitud, longitud, altitud y radio de precisión), los cuales varían estructuralmente dependiendo de la API del sistema operativo (Android o iOS) o del sensor del hardware. Utilizar un tipo relacional rígido obligaría a realizar constantes modificaciones al esquema general (ALTER TABLE) o a almacenar cientos de columnas vacías (valores NULL), destruyendo el rendimiento del disco duro. El tipo NoSQL concentrado absorbe esta variabilidad de forma nativa.

Caso de Uso 2: Big Data Analítico y Telemetría de Navegación Estudiantil
El éxito de una red social depende en gran medida de la velocidad con la que se conectan los estudiantes según su necesidad y afinidades. Para proveer datos a modelos analíticos inteligentes en la tabla perfil se introduce un campo NoSQL flexible llamado auditoria_navegacion. Este campo acapara las cantidades masivas de la huella digital de los estudiantes en cada sesión activa; direcciones IP, metadatos del navegador web, resolución de pantalla sistema operativo windows utilizado, e historial de clics lúdicos. Acumular este enorme volumen de información estructurada en tablas relacionales tradicionales provocaría un crecimiento exponencial de registros interconectados, ralentizando las operaciones de base de la aplicación mediante costosos bloqueos de fila. Al encapsular estos datos en documentos semiestructurados indexados en formato binario, se habilita la extracción masiva hacia repositorios de Big Data sin penalizar el rendimiento del núcleo transaccional.

2. ANÁLISIS COMPARATIVO DE SISTEMAS GESTORES (SGBD)

Criterio Técnico	PostgreSQL (Recomendado)	MySQL (Variante)	SQL Server (Variante)
Manejo de Datos NoSQL / JSON	Excelente. Soporta tipo JSONB que descompone los datos en formato binario eficiente permitiendo indexación GIN avanzada.	Bueno. Ofrece tipo JSON nativo con validación automática y funciones de extracción, pero con menor madurez en indexación profunda.	Aceptable. No posee tipo de dato JSON nativo independiente; utiliza VARCHAR(MAX) con restricciones de validación ISJSON.
Tipo de datos para textos largos	Excelente rendimiento mediante la técnica nativa TOAST, aislando textos extensos en bloques independientes sin penalizar filas primarias.	Bueno mediante tipos TEXT/LONGTEXT, pero con penalizaciones en velocidad de lectura cuando ocurren JOIN cruzados masivos.	Muy bueno utilizando VARCHAR(MAX), integrando compresión eficiente de páginas de datos en sistemas robustos.
Concurrencia y Alta Carga	Sobresaliente. Utiliza control multiversión MVCC. Evita bloqueos entre lecturas y escrituras cuando los estudiantes reaccionan en masa.	Eficiente. Emplea el motor InnoDB con bloqueo a nivel de fila, sufriendo leves degradaciones ante escrituras simultáneas masivas.	Excelente en entornos corporativos utilizando Aislamiento Snapshot, requiriendo un alto consumo de recursos de hardware y RAM.
Escalabilidad Big Data / IoT	Máxima. La combinación de JSONB, índices GIN e índices parciales permite realizar búsquedas instantáneas sobre millones de filas.	Media. Adecuado para lecturas operacionales rápidas, pero limitado en análisis de agregación geoespacial compleja semiestructurada.	Alta. Cuenta con soporte Columnstore integrado, requiriendo licencias costosas para explotar el rendimiento máximo.

3. DICCIONARIO FISICO COMPARATIVO

Tabla 1: estudiante (datos de cuenta de identidad)

Nombre Campo	**PostgreSQL**	**MySQL**	**SQL Server**	**Restricciones / Llaves**	**Descripción**
id_estudiante	SERIAL	INT AUTO_INCREMENT	INT IDENTITY (1,1)	NOT NULL, PK	Identificador único secuencial.
nombre	VARCHAR (100)	VARCHAR (100)	VARCHAR (100)	NOT NULL	Nombre completo del alumno.
correo	VARCHAR (150)	VARCHAR (150)	VARCHAR (150)	NOT NULL, UNIQUE	Correo institucional de acceso.
password_hash	VARCHAR (255)	VARCHAR (255)	VARCHAR (255)	NOT NULL	Clave cifrada por seguridad.
fecha_registro	TIMESTAMP	TIMESTAMP	DATETIME	DEFAULT Activa	Fecha automática de registro.

Tabla 2: perfil (información social y telemetría Big Data)

Nombre Campo	**PostgreSQL**	**MySQL**	**SQL Server**	**Restricciones / Llaves**	**Descripción**
id_perfil	SERIAL	INT AUTO_INCREMENT	INT IDENTITY (1,1)	NOT NULL, PK	ID único del perfil social.
id_estudiante	INT	INT	INT	NOT NULL, FK (Cascada)	Relación 1:1 con Estudiante.
biografía	TEXT	TEXT	VARCHAR(MAX)	NULL	Descripción libre del estudiante.
Auditoria_navegacion	JSONB	JSON	VARCHAR(MAX)	NULL / CHECK ISJSON	Documento NoSQL con la telemetría e IP del dispositivo.

Tabla 3: evento (coordinación y datos IoT remotos)

Nombre Campo	PostgreSQL	MySQL	SQL Server	Restricciones / Llaves	Descripción
id_evento	SERIAL	INT AUTO_INCREMENT	INT IDENTITY(1,1)	NOT NULL, PK	ID único del evento.
nombre	VARCHAR (100)	VARCHAR (100)	VARCHAR (100)	NOT NULL	Nombre de la actividad.
geolocalización	JSONB	JSON	VARCHAR (MAX)	NULL / CHECK ISJSON	Coordenadas IoT de sensores localización en mapa.

Tabla 4: publicación

Nombre Campo	PostgreSQL	MySQL	SQL Server	Restricciones / Llaves	Descripción
id_publicacion	SERIAL	INT AUTO_INCREMENT	INT IDENTITY (1,1)	NOT NULL, PK	Identificador único de la publicación.
id_estudiante	INT	INT	INT	NOT NULL, FK	Autor de la publicación.
Id_evento	INT	INT	INT	NULL, FK	Identificador único del evento asociado a la publicación (Clave foránea opcional hacia la tabla evento).
contenido	TEXT	TEXT	VARCHAR(MAX)	NOT NULL	Texto o contenido compartido.
fecha_publicacion	TIMESTAMP	TIMESTAMP	DATETIME	DEFAULT actual	Fecha de creación de la publicación.

Tabla 5: Grupo (Comunidades de Estudio)

Nombre Campo	PostgreSQL	MySQL	SQL Server	Restricciones / Llaves	Descripción
id_grupo	SERIAL	INT AUTO_INCREMENT	INT IDENTITY(1,1)	NOT NULL, PK	Identificador único del grupo.
nombre	VARCHAR (100)	VARCHAR (100)	VARCHAR (100)	NOT NULL	Nombre del grupo o comunidad.
descripción	TEXT	TEXT	VARCHAR(MAX)	NULL	Descripción del objetivo del grupo.
id_creador	INT	INT	INT	NOT NULL, FK	Estudiante que creó el grupo.

Tabla 6: miembro grupo (Asociación N:M)

Nombre Campo	PostgreSQL	MySQL	SQL Server	Restricciones / Llaves	Descripción
id_estudiante	INT	INT	INT	PK, FK	Estudiante miembro del grupo.
id_grupo	INT	INT	INT	PK, FK	Grupo al que pertenece.
fecha_union	DATE	DATE	DATE	NULL	Fecha de ingreso al grupo.

4. SCRIPTS DDL DE IMPLEMENTACIÓN FÍSICA MULTI-MOTOR

Implementación Avanzada en PostgreSQL (Motor Principal Relacional-Objeto)

```
-- ========================================
-- 1. TABLA ESTUDIANTE
-- ========================================
CREATE TABLE estudiante (
   id_estudiante SERIAL,
   nombre VARCHAR(100) NOT NULL,
   correo VARCHAR(150) NOT NULL UNIQUE,
   password_hash VARCHAR(255) NOT NULL,
   fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT pk_estudiante
   PRIMARY KEY (id_estudiante)
);

-- ========================================
-- 2. TABLA PERFIL
-- ========================================
CREATE TABLE perfil (
```

```
    id_perfil SERIAL,
    id_estudiante INT NOT NULL,
    biografia TEXT,
    auditoria_navegacion JSONB,
    CONSTRAINT pk_perfil
    PRIMARY KEY (id_perfil),
    CONSTRAINT fk_perfil_estudiante
    FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE CASCADE
);

-- ========================================
-- 3. TABLA EVENTO
-- ========================================
CREATE TABLE evento (
    id_evento SERIAL,
    nombre VARCHAR(100) NOT NULL,
    geolocalizacion JSONB,
    CONSTRAINT pk_evento
    PRIMARY KEY (id_evento)
);

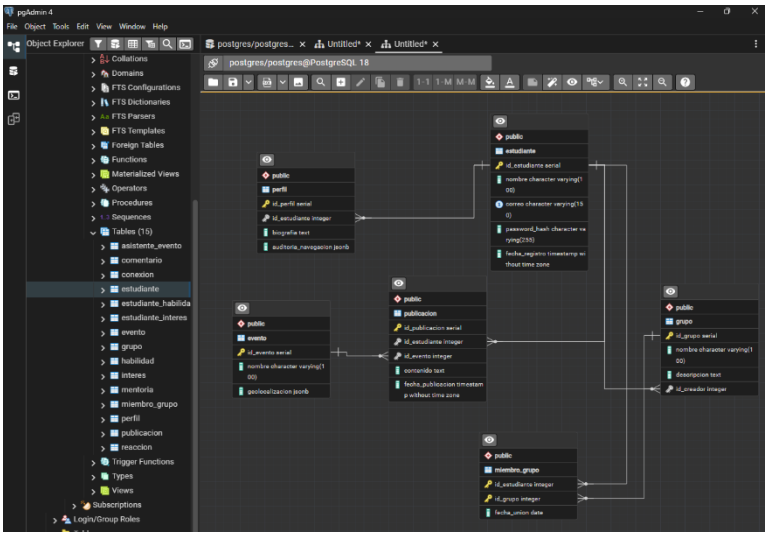
-- ========================================
-- 4. TABLA PUBLICACION
-- ========================================
CREATE TABLE publicacion (
    id_publicacion SERIAL,
    id_estudiante INT NOT NULL,
    id_evento INT, -- Se añade la columna para conectar con evento (puede ser NULL
si una publicación no es de un evento)
    contenido TEXT NOT NULL,
    fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

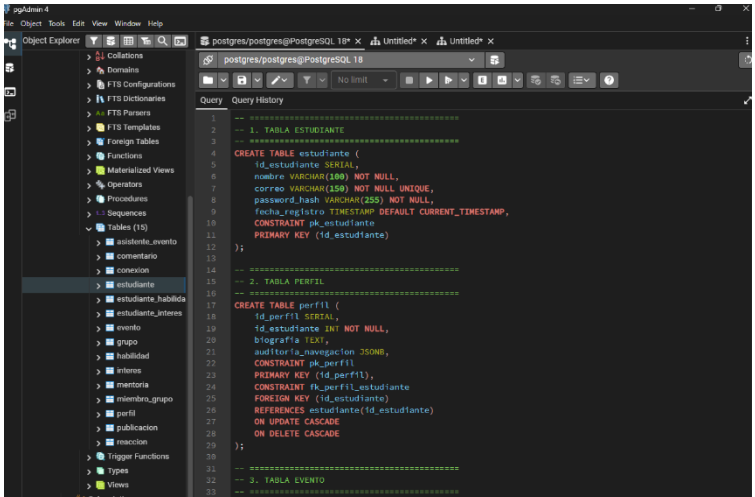
```
    CONSTRAINT pk_publicacion
    PRIMARY KEY (id_publicacion),
    CONSTRAINT fk_publicacion_estudiante
    FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
    CONSTRAINT fk_publicacion_evento
    FOREIGN KEY (id_evento)
    REFERENCES evento(id_evento) -- Crea la llave foránea física hacia la tabla evento
    ON UPDATE CASCADE
    ON DELETE SET NULL
);

-- ========================================
-- 5. TABLA GRUPO
-- ========================================
CREATE TABLE grupo (
    id_grupo SERIAL,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    id_creador INT NOT NULL,
    CONSTRAINT pk_grupo
    PRIMARY KEY (id_grupo),
    CONSTRAINT fk_grupo_estudiante
    FOREIGN KEY (id_creador)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

-- ========================================
-- 6. TABLA MIEMBRO_GRUPO
-- ========================================
```

```
CREATE TABLE miembro_grupo (
    id_estudiante INT NOT NULL,
    id_grupo INT NOT NULL,
    fecha_union DATE DEFAULT CURRENT_DATE,
    CONSTRAINT pk_miembro_grupo
    PRIMARY KEY (id_estudiante, id_grupo),
    CONSTRAINT fk_miembro_grupo_estudiante
    FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
    CONSTRAINT fk_miembro_grupo_grupo
    FOREIGN KEY (id_grupo)
    REFERENCES grupo(id_grupo)
    ON UPDATE CASCADE
    ON DELETE CASCADE
);
```





Implementación Adaptada en MySQL (Motor Relacional Puro)

```
-- ========================================
-- 1. TABLA ESTUDIANTE
-- ========================================
CREATE TABLE estudiante (
    id_estudiante INT AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(150) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_estudiante
    PRIMARY KEY (id_estudiante),
    CONSTRAINT uq_estudiante_correo
    UNIQUE (correo)
);

-- ========================================
-- 2. TABLA PERFIL
-- ========================================
CREATE TABLE perfil (
    id_perfil INT AUTO_INCREMENT,
    id_estudiante INT NOT NULL,
    biografia TEXT,
    auditoria_navegacion JSON, -- Tipo JSON estándar en MySQL
    CONSTRAINT pk_perfil
    PRIMARY KEY (id_perfil),
    CONSTRAINT fk_perfil_estudiante
```

```
    FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE CASCADE
);

-- ========================================
-- 3. TABLA EVENTO
-- ========================================
CREATE TABLE evento (
    id_evento INT AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    geolocalizacion JSON, -- Tipo JSON estándar en MySQL
    CONSTRAINT pk_evento
    PRIMARY KEY (id_evento)
);

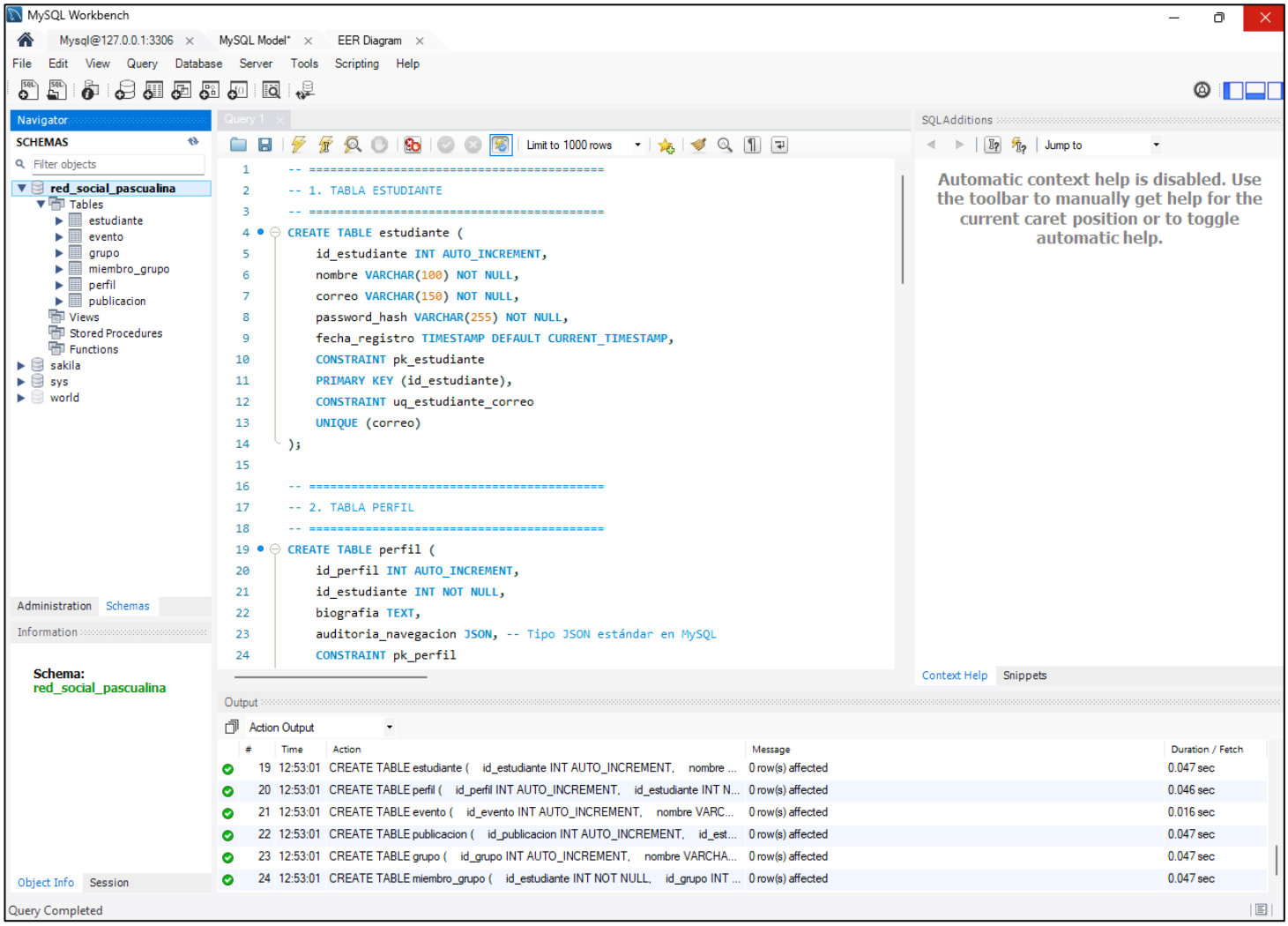
-- ========================================
-- 4. TABLA PUBLICACION
-- ========================================
CREATE TABLE publicacion (
    id_publicacion INT AUTO_INCREMENT,
    id_estudiante INT NOT NULL,
    id_evento INT NULL, -- Puede ser NULL si una publicación no pertenece a un evento
    contenido TEXT NOT NULL,
    fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_publicacion
    PRIMARY KEY (id_publicacion),
    CONSTRAINT fk_publicacion_estudiante
    FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
```

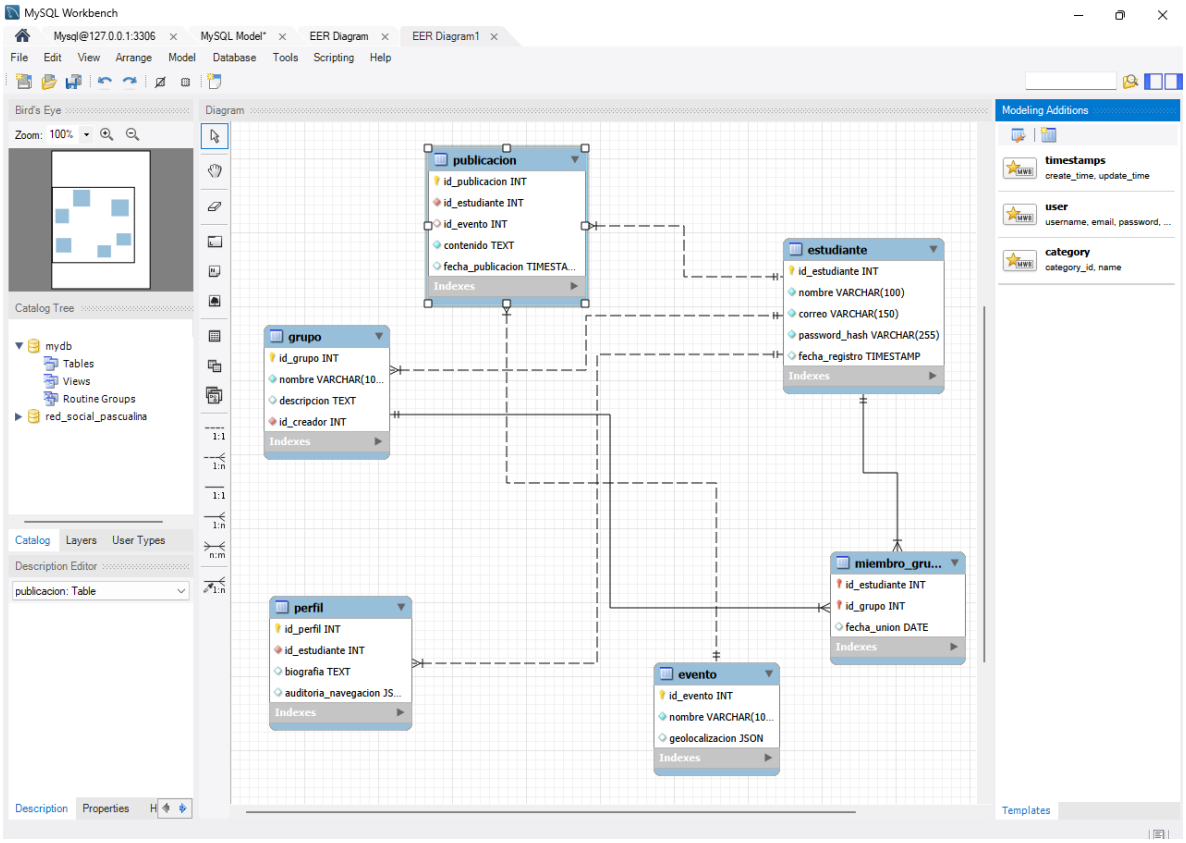
```
    CONSTRAINT fk_publicacion_evento
    FOREIGN KEY (id_evento)
    REFERENCES evento(id_evento)
    ON UPDATE CASCADE
    ON DELETE SET NULL
);


-- ========================================
-- 5. TABLA GRUPO
-- ========================================
CREATE TABLE grupo (
    id_grupo INT AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    id_creador INT NOT NULL,
    CONSTRAINT pk_grupo
    PRIMARY KEY (id_grupo),
    CONSTRAINT fk_grupo_estudiante
    FOREIGN KEY (id_creador)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

-- ========================================
-- 6. TABLA MIEMBRO_GRUPO
-- ========================================
CREATE TABLE miembro_grupo (
    id_estudiante INT NOT NULL,
    id_grupo INT NOT NULL,
    fecha_union DATE DEFAULT (CURRENT_DATE), -- Sintaxis compatible con MySQL
moderno
    CONSTRAINT pk_miembro_grupo
```

```
    PRIMARY KEY (id_estudiante, id_grupo),
    CONSTRAINT fk_miembro_grupo_estudiante
    FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
    CONSTRAINT fk_miembro_grupo_grupo
    FOREIGN KEY (id_grupo)
    REFERENCES grupo(id_grupo)
    ON UPDATE CASCADE
    ON DELETE CASCADE
);
```




MySQL Workbench
Mysql@127.0.0.1:3306
MySQL Model*
EER Diagram
EER Diagram1
File Edit View Arrange Model Database Tools Scripting Help
Bird's Eye
Zoom: 100%
Catalog Tree
mydb
Tables
Views
Routine Groups
red_social_pascualina
Catalog Layers User Types
Description Editor
publicacion: Table
Description Properties
Diagram
publicacion
id_publicacion INT
id_estudiante INT
id_evento INT
contenido TEXT
fecha_publicacion TIMESTA...
Indexes
estudiante
id_estudiante INT
nombre VARCHAR(100)
correo VARCHAR(150)
password_hash VARCHAR(255)
fecha_registro TIMESTAMP
Indexes
grupo
id_grupo INT
nombre VARCHAR(10...
descripcion TEXT
id_creador INT
Indexes
miembro_gru...
id_estudiante INT
id_grupo INT
fecha_union DATE
Indexes
perfil
id_perfil INT
id_estudiante INT
biografia TEXT
auditoria_navegacion JS...
Indexes
evento
id_evento INT
nombre VARCHAR(10...
geolocalizacion JSON
Indexes
Modeling Additions
timestamps
create_time, update_time
user
username, email, password, ...
category
category_id, name
Templates

Implementación Adaptada en SQL Server

```
-- ========================================
-- 1. TABLA ESTUDIANTE
-- ========================================
CREATE TABLE estudiante (
    id_estudiante INT IDENTITY(1,1),
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(150) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    fecha_registro DATETIME DEFAULT GETDATE(),

    CONSTRAINT pk_estudiante
    PRIMARY KEY (id_estudiante),

    CONSTRAINT uq_estudiante_correo
    UNIQUE (correo)
);

-- ========================================
-- 2. TABLA PERFIL
-- ========================================
CREATE TABLE perfil (
    id_perfil INT IDENTITY(1,1),
    id_estudiante INT NOT NULL,
    biografia TEXT,
    auditoria_navegacion NVARCHAR(MAX), -- Tipo recomendado para guardar JSON en
SQL Server

    CONSTRAINT pk_perfil
    PRIMARY KEY (id_perfil),
```

```
    CONSTRAINT fk_perfil_estudiante
    FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE CASCADE,

    -- Restricción para asegurar que solo se guarde JSON válido
    CONSTRAINT chk_perfil_auditoria
    CHECK (ISJSON(auditoria_navegacion) > 0)
);

-- ========================================
-- 3. TABLA EVENTO
-- ========================================
CREATE TABLE evento (
    id_evento INT IDENTITY(1,1),
    nombre VARCHAR(100) NOT NULL,
    geolocalizacion NVARCHAR(MAX), -- Tipo recomendado para guardar JSON en SQL Server

    CONSTRAINT pk_evento
    PRIMARY KEY (id_evento),

    -- Restricción para asegurar que solo se guarde JSON válido
    CONSTRAINT chk_evento_geolocalizacion
    CHECK (ISJSON(geolocalizacion) > 0)
);

-- ========================================
-- 4. TABLA PUBLICACION
-- ========================================
CREATE TABLE publicacion (
    id_publicacion INT IDENTITY(1,1),
    id_estudiante INT NOT NULL,
```

```
    id_evento INT NULL, -- Puede ser NULL si una publicación no pertenece a un evento
    contenido TEXT NOT NULL,
    fecha_publicacion DATETIME DEFAULT GETDATE(),

    CONSTRAINT pk_publicacion
    PRIMARY KEY (id_publicacion),

    CONSTRAINT fk_publicacion_estudiante
    FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE CASCADE
    ON DELETE CASCADE,

    CONSTRAINT fk_publicacion_evento
    FOREIGN KEY (id_evento)
    REFERENCES evento(id_evento)
    ON UPDATE NO ACTION -- SQL Server requiere NO ACTION para evitar múltiples rutas en cascada
    ON DELETE SET NULL
);

-- ========================================
-- 5. TABLA GRUPO
-- ========================================
CREATE TABLE grupo (
    id_grupo INT IDENTITY(1,1),
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    id_creador INT NOT NULL,

    CONSTRAINT pk_grupo
    PRIMARY KEY (id_grupo),

    CONSTRAINT fk_grupo_estudiante
```

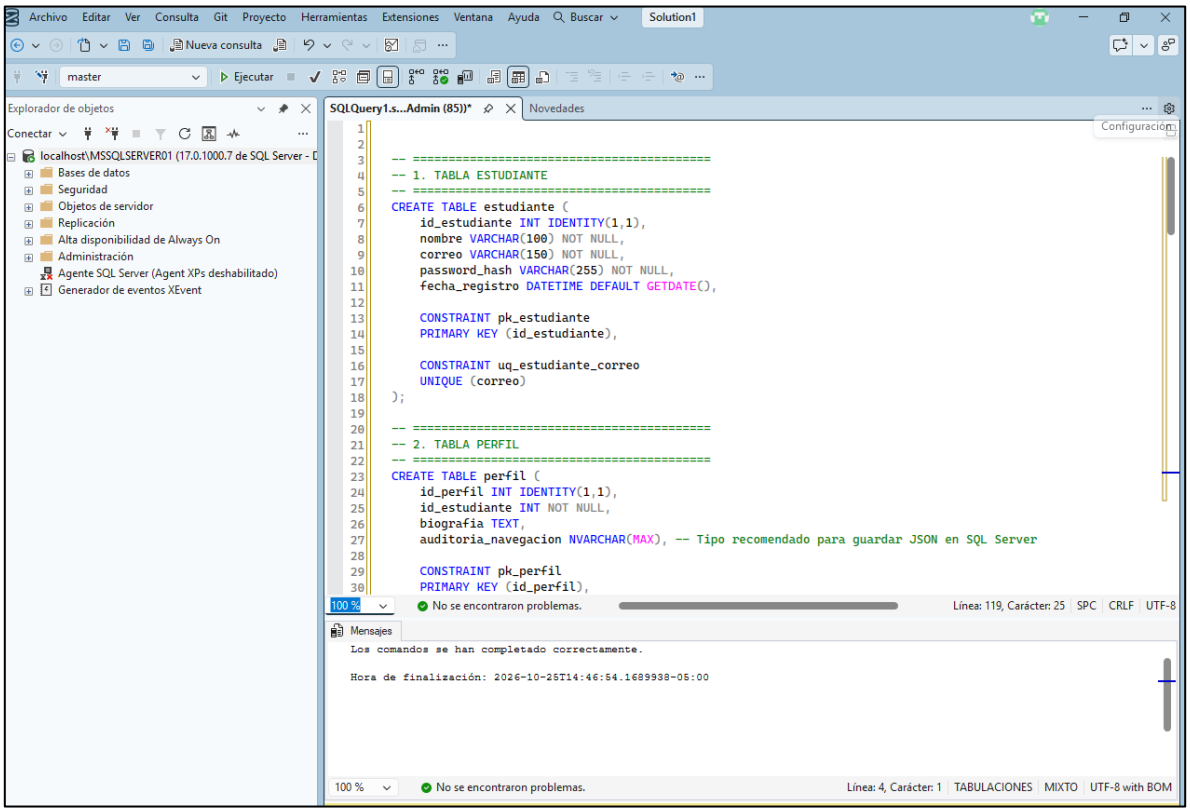
```
    FOREIGN KEY (id_creador)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
);

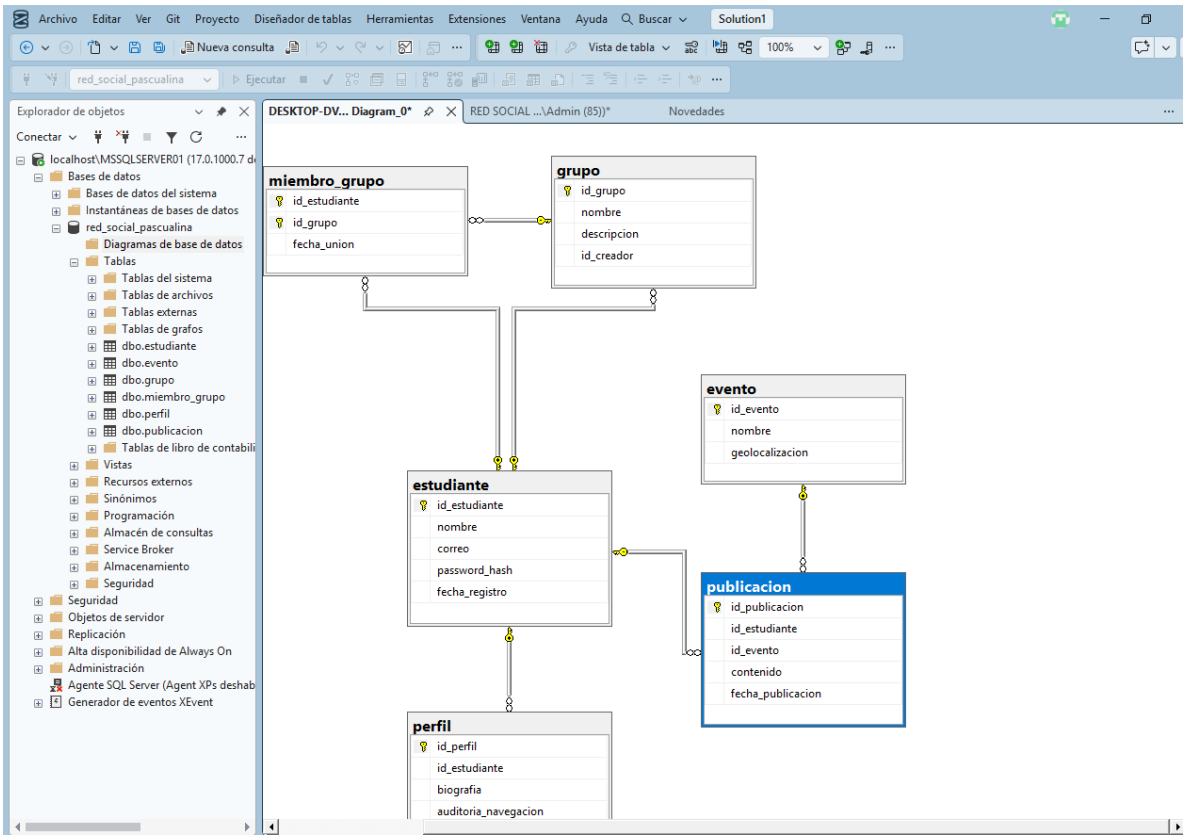
-- ========================================
-- 6. TABLA MIEMBRO_GRUPO
-- ========================================
CREATE TABLE miembro_grupo (
    id_estudiante INT NOT NULL,
    id_grupo INT NOT NULL,
    fecha_union DATE DEFAULT CAST(GETDATE() AS DATE),

    CONSTRAINT pk_miembro_grupo
    PRIMARY KEY (id_estudiante, id_grupo),

    CONSTRAINT fk_miembro_grupo_estudiante
    FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante)
    ON UPDATE NO ACTION
    ON DELETE NO ACTION,

    CONSTRAINT fk_miembro_grupo_grupo
    FOREIGN KEY (id_grupo)
    REFERENCES grupo(id_grupo)
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
);
```


Archivo Editar Ver Consulta Git Proyecto Herramientas Extensiones Ventana Ayuda Buscar Solution1
Nueva consulta
master
Ejecutar
Explorador de objetos
Conectar
localhost\MSSQLSERVER01 (17.0.1000.7 de SQL Server - D
Bases de datos
Seguridad
Objetos de servidor
Replicación
Alta disponibilidad de Always On
Administración
Agente SQL Server (Agent XPs deshabilitado)
Generador de eventos XEvent
SQLQuery1.s...Admin (85))*
Novedades
Configuración
-- ==
-- 1. TABLA ESTUDIANTE
-- ==
CREATE TABLE estudiante (
id_estudiante INT IDENTITY(1,1),
nombre VARCHAR(100) NOT NULL,
correo VARCHAR(150) NOT NULL,
password_hash VARCHAR(255) NOT NULL,
fecha_registro DATETIME DEFAULT GETDATE(),
CONSTRAINT pk_estudiante
PRIMARY KEY (id_estudiante),
CONSTRAINT uq_estudiante_correo
UNIQUE (correo)
);
-- ==
-- 2. TABLA PERFIL
-- ==
CREATE TABLE perfil (
id_perfil INT IDENTITY(1,1),
id_estudiante INT NOT NULL,
biografia TEXT,
auditoria_navegacion NVARCHAR(MAX), -- Tipo recomendado para guardar JSON en SQL Server
CONSTRAINT pk_perfil
PRIMARY KEY (id_perfil),
100 %
No se encontraron problemas.
Línea: 119, Carácter: 25
SPC
CRLF
UTF-8
Mensajes
Los comandos se han completado correctamente.
Hora de finalización: 2026-10-25T14:46:54.1689938-05:00
100 %
No se encontraron problemas.
Línea: 4, Carácter: 1
TABULACIONES
MIXTO
UTF-8 with BOM



DIFICULTADES ENCONTRADAS, VENTAJAS Y DESVENTAJAS OBSERVADAS EN CADA CASO.

1. Dificultades durante la instalación y configuración

SQL Server: Fue el sistema que presentó mayores dificultades durante la instalación y configuración. Inicialmente, seleccionamos la versión Evaluation, que tiene un período de uso limitado. Posteriormente, decidimos descargar la versión Developer, que es gratuita y está orientada al desarrollo y aprendizaje. También comprendimos que era necesario instalar dos componentes por separado: el motor de base de datos y SQL Server Management Studio (SSMS), que permite administrar las bases de datos mediante una interfaz gráfica.

Al intentar conectarnos, encontramos un inconveniente relacionado con la seguridad y el cifrado de la conexión. Para solucionarlo, habilitamos la opción «Confiar en el certificado del servidor», lo que nos permitió continuar con el trabajo. Esta experiencia nos permitió reconocer que SQL Server ofrece mecanismos de seguridad importantes, aunque su configuración inicial puede resultar compleja para quienes comienzan a utilizarlo.

MySQL: La instalación de MySQL fue más sencilla en comparación con SQL Server. Sin embargo, encontramos dificultades al ejecutar nuevamente el script SQL después de realizar modificaciones. El sistema mostraba errores indicando que algunas tablas, como estudiante, ya existían, lo que impedía continuar con la ejecución del código. Para resolver este inconveniente, aprendimos a utilizar la instrucción DROP TABLE IF EXISTS, que permite eliminar las tablas existentes antes de crearlas nuevamente. Esto nos ayudó a comprender la importancia de controlar las dependencias y el orden de ejecución de los scripts.

PostgreSQL: La instalación se realizó correctamente, aunque durante el uso de pgAdmin surgieron algunas dificultades relacionadas con su interfaz gráfica. En ocasiones, cerrábamos accidentalmente la pestaña del editor SQL y la ventana de trabajo quedaba vacía, lo que nos hacía pensar que habíamos perdido el código. Posteriormente, identificamos que podíamos recuperar el espacio de trabajo haciendo clic derecho sobre la base de datos y seleccionando la herramienta *Query Tool*. Aunque al principio la interfaz nos resultó poco intuitiva, aprendimos a utilizar mejor sus herramientas de administración.

2. Dificultades durante la implementación del código y la elaboración de los diagramas.

Problemas con las relaciones entre las tablas: Durante la elaboración de los diagramas entidad-relación, encontramos dificultades para establecer correctamente algunas conexiones. En particular, la tabla evento aparecía sin relacionarse con las demás tablas o las líneas de conexión se generaban de manera incorrecta. Después de revisar el código, identificamos que faltaba el campo id_evento en la tabla publicacion, necesario para establecer la relación correspondiente. Al agregar esta columna y definir correctamente la clave foránea, logramos representar las relaciones de manera adecuada. Esta situación nos enseñó que los diagramas dependen de una estructura bien definida y de la correcta implementación de las relaciones en el código.

Restricciones de eliminación en cascada en SQL Server (error 1785): Uno de los inconvenientes más complejos se presentó al utilizar la instrucción ON DELETE CASCADE en varias relaciones. SQL Server restringe determinadas configuraciones de eliminación en cascada cuando generan múltiples rutas de eliminación hacia una misma tabla. En nuestro caso, las relaciones de las tablas publicacion y miembro_grupo con estudiante provocaron el error 1785. Para solucionarlo, modificamos las restricciones correspondientes y utilizamos ON DELETE NO ACTION, evitando eliminaciones automáticas que entraran en

conflicto con otras relaciones. Aunque esta restricción representó una dificultad, también nos permitió comprender la importancia de mantener la integridad de los datos.

Diferencias en el manejo de datos JSON: Como parte de los requisitos del proyecto, incorporamos datos en formato JSON para representar información semiestructurada, como las direcciones IP de los perfiles y las coordenadas GPS de los eventos, relacionadas con conceptos de Big Data e Internet de las Cosas (IoT). Durante la implementación, identificamos diferencias entre los tres sistemas gestores. PostgreSQL dispone del tipo JSONB, que almacena los datos en un formato binario optimizado; MySQL ofrece un tipo de dato JSON nativo; mientras que SQL Server permite almacenar este contenido como texto mediante NVARCHAR(MAX) y validar su formato con funciones como ISJSON. Estas diferencias nos obligaron a adaptar el código según las características de cada plataforma.

3. Ventajas y desventajas de los sistemas gestores de bases de datos

SQL Server: Entre sus principales ventajas se encuentran las herramientas gráficas de administración, las opciones de seguridad y los mecanismos para controlar la integridad de los datos. Además, permite desarrollar y administrar bases de datos mediante SQL Server Management Studio, lo que facilita la supervisión de las tablas y sus relaciones.

Como desventajas, su instalación puede requerir varios pasos y su configuración inicial puede resultar compleja. Asimismo, presenta restricciones específicas en algunas operaciones, como las eliminaciones en cascada, que obligan a adaptar el código según sus reglas.

MySQL: Una de sus principales ventajas es que ofrece una instalación relativamente sencilla y dispone de un tipo de dato JSON nativo, útil para almacenar información semiestructurada. También permite crear y relacionar tablas mediante instrucciones SQL, lo que facilita el desarrollo de aplicaciones que necesitan organizar grandes cantidades de información.

Entre sus desventajas, la ejecución repetida de scripts puede generar errores cuando las tablas ya existen o cuando hay dependencias entre ellas. Por esta razón, es necesario controlar el orden de creación y eliminación de las tablas para evitar conflictos durante las pruebas.

PostgreSQL:

Entre sus ventajas se destaca el soporte de funciones avanzadas para el manejo de datos, especialmente mediante JSONB, así como la posibilidad de establecer relaciones, restricciones y validaciones que contribuyen a mantener la integridad de la información. Estas características lo convierten en una alternativa flexible para proyectos que requieren estructuras relacionales y datos semiestructurados.

Como desventajas, algunas funciones de pgAdmin pueden resultar poco intuitivas para los usuarios principiantes. Además, el uso de sus características avanzadas requiere familiarización con las herramientas y los tipos de datos disponibles.

4. Aprendizajes obtenidos

El desarrollo del proyecto nos permitió comprender que SQL Server, MySQL y PostgreSQL cumplen funciones similares, pero presentan diferencias en su instalación, administración, sintaxis y manejo de restricciones. Cada sistema ofrece ventajas que pueden resultar útiles según las necesidades de un proyecto, así como limitaciones que deben tenerse en cuenta durante su implementación.

A través de las dificultades encontradas, fortalecimos nuestros conocimientos sobre la configuración de entornos de trabajo, la creación de tablas, el establecimiento de claves foráneas, las restricciones de integridad y el manejo de datos JSON. También aprendimos a identificar errores, analizar sus causas y adaptar las instrucciones SQL a las particularidades de cada plataforma.

CONCLUSIONES

SONIA: Me permitió comprender de manera práctica el impacto directo de un correcto diseño físico en el rendimiento de una arquitectura transaccional multi-motor. Enfrentar y solucionar autónomamente las restricciones de clave foránea cruzadas nos otorgaron una perspectiva del mundo real sobre los desafíos que asume un administrador de sistemas. El enfoque híbrido SQL + NoSQL es ideal para las dinámicas estudiantiles modernas, pues absorbe flujos geoespaciales inestables (IoT) sin degradar la normalización relacional. La sintaxis estricta de PostgreSQL y la versatilidad de su almacenamiento binario JSONB me dio una visión clara de cómo los requerimientos de un usuario, como gestionar hackáthones o registrar la huella digital del navegador, se transforman en estructuras eficientes.

Aprendí a no ver los errores de relación como trabas, sino como mecanismos de protección de los datos.

CAMILA: La comparación de estos tres sistemas gestores me permitió reconocer que la elección de una herramienta no depende únicamente de su facilidad de instalación, sino también de sus funcionalidades, requisitos técnicos y capacidad para responder a las necesidades del proyecto. Esta experiencia contribuyó a nuestro aprendizaje práctico y al desarrollo de habilidades para diseñar e implementar bases de datos de manera más organizada y eficiente.